

Complete Kubernetes Microservices Guide with Monitoring & Backup

This comprehensive guide will take you from zero to a production-ready Kubernetes microservices platform. You'll build:

- **Container Orchestration:** EKS (Amazon Elastic Kubernetes Service) or Minikube cluster
- **Package Management:** Helm charts for easy deployment
- **Monitoring Stack:** Prometheus + Grafana with custom dashboards
- **Backup Solution:** Velero for disaster recovery
- **Auto-scaling:** HPA (Horizontal Pod Autoscaler) & VPA (Vertical Pod Autoscaler) for dynamic resource management
- **Production-ready configurations** with alerting and security

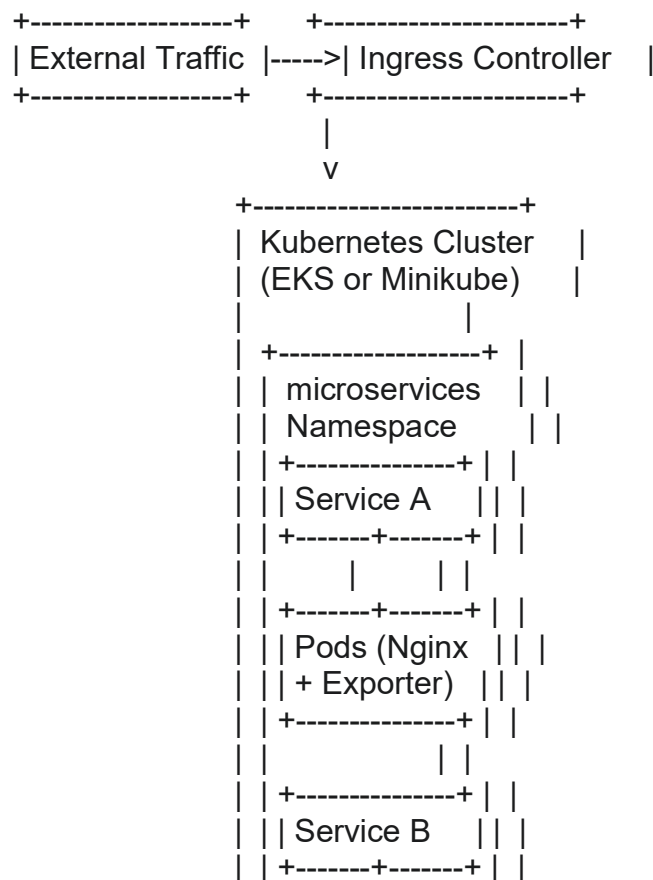
Time Required: 4-6 hours for complete setup

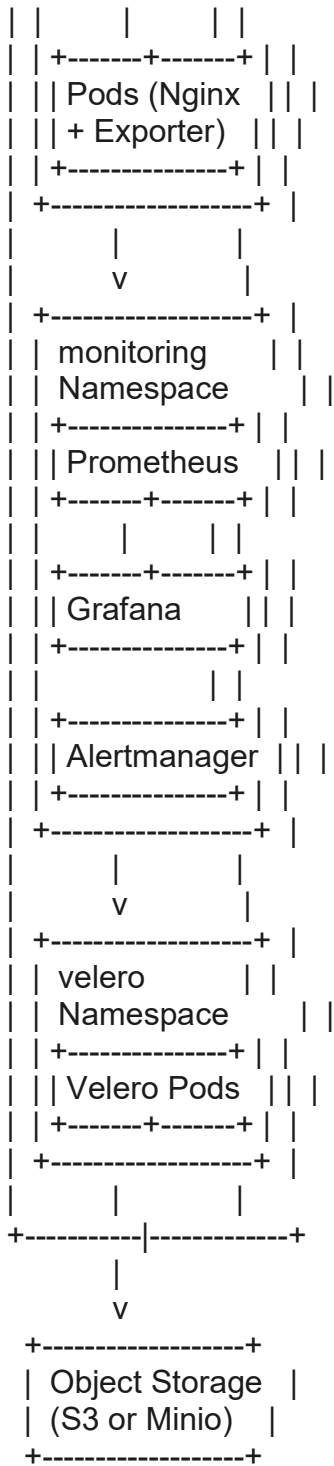
Skill Level: Beginner (with detailed explanations)

Cost: Free with Minikube, ~\$50-100/month with EKS (depending on usage)

Project Architecture Overview

Here's a high-level view of the components we'll be setting up and how they interact:





Prerequisites and Environment Setup

Before we begin, make sure your system meets these requirements and you have a basic understanding of some key concepts.

Required Knowledge

- Basic command line usage
- Understanding of containers and Docker basics
- Basic YAML syntax awareness

System Requirements

- **OS:** Linux (Ubuntu 20.04+ recommended) or macOS
- **RAM:** Minimum 8GB, recommended 16GB
- **Disk:** 50GB free space
- **CPU:** 4+ cores recommended

Step 1: Install Required Tools (30 minutes)

We'll start by installing all the necessary command-line tools.

1.1 Install Docker (Ubuntu/Debian)

Docker is essential for building and running containers.

```
# Update package index
sudo apt update
```

```
# Install prerequisite packages
sudo apt install -y apt-transport-https ca-certificates curl software-properties-common
```

```
# Add Docker's official GPG key
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/docker-archive-keyring.gpg
```

```
# Add Docker repository
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-
archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs)
stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
# Install Docker
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io
```

```
# Add your user to the docker group so you don't need sudo for docker commands
sudo usermod -aG docker $USER
```

```
# Log out and back in (or restart your terminal) for the group change to take effect, then
test Docker
docker --version
```

1.2 Install kubectl

kubectl is the command-line tool for interacting with Kubernetes clusters.

```
# Download the latest kubectl binary for Linux AMD64
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"

# Make it executable
chmod +x kubectl

# Move it to a directory in your system PATH (e.g., /usr/local/bin/)
sudo mv kubectl /usr/local/bin/

# Verify installation
kubectl version --client
```

1.3 Install Helm

Helm is the package manager for Kubernetes, making it easy to deploy and manage applications.

```
# Download Helm installation script
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-
helm-3

# Make script executable
chmod 700 get_helm.sh

# Run installation script
./get_helm.sh

# Verify installation
helm version
```

1.4 Install Minikube (for local development)

Minikube allows you to run a single-node Kubernetes cluster on your local machine. It's great for learning and development.

```
# Download Minikube binary for Linux AMD64
```

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
```

```
# Install Minikube
```

```
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

```
# Verify installation
```

```
minikube version
```

1.5 Optional: Install AWS CLI (for EKS)

If you plan to use Amazon EKS for your Kubernetes cluster, you'll need the AWS Command Line Interface.

```
# Download AWS CLI installer
```

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
```

```
# Install unzip if not present
```

```
sudo apt install -y unzip
```

```
# Extract and install
```

```
unzip awscliv2.zip
```

```
sudo ./aws/install
```

```
# Verify installation
```

```
aws --version
```

```
# Configure AWS credentials (if using EKS)
```

```
# This will prompt you for your AWS Access Key ID, Secret Access Key, default region, and output format.
```

```
# Make sure you have an AWS account and IAM user with appropriate permissions.
```

```
aws configure
```

1.6 Create Project Directory Structure

Let's set up a clean directory structure for our project files.

```
# Create main project directory
mkdir -p ~/kubernetes-microservices
cd ~/kubernetes-microservices

# Create subdirectories for Helm charts, Kubernetes manifests, monitoring
configurations, and backup files
mkdir -p helm/{microservice-a,microservice-b,monitoring}
mkdir -p k8s/{namespaces,rbac,ingress,autoscaling,testing,quotas,network}
mkdir -p monitoring/{prometheus,grafana>alertmanager}
mkdir -p backup/{velero,schedules}

# Verify structure (install 'tree' if you don't have it: sudo apt install tree)
tree . || find . -type d
```

Phase 1: Kubernetes Cluster Setup (45 minutes)

You have two options for setting up your Kubernetes cluster: Minikube for local development or EKS for a production-ready cloud environment. Choose one based on your needs.

Option A: Minikube Setup (Recommended for Beginners)

Step 1.1: Start Minikube with Adequate Resources

It's crucial to allocate enough resources to your Minikube cluster for our services to run smoothly.

```
# Start Minikube with sufficient resources for our project
minikube start \
  --cpus=4 \
  --memory=8192mb \
  --disk-size=50g \
  --driver=docker \
  --kubernetes-version=v1.28.0

# Wait for startup (this may take 5-10 minutes on first run as it downloads images)
```

Step 1.2: Enable Required Add-ons

Minikube comes with several useful add-ons. We'll enable the ingress controller (for routing external traffic) and metrics-server (needed for autoscaling). The dashboard is optional but helpful for visual inspection.

```
# Enable ingress controller
minikube addons enable ingress
echo "✓ Ingress controller enabled"
```

```
# Enable metrics server (required for HPA)
minikube addons enable metrics-server
echo "✓ Metrics server enabled"
```

```
# Enable dashboard (optional but helpful for visual management)
minikube addons enable dashboard
echo "✓ Dashboard enabled"
```

```
# Verify cluster is running and kubectl is configured
kubectl cluster-info
```

Step 1.3: Verify Cluster Health

Always check if your cluster nodes and system pods are running correctly.

```
# Check node status
kubectl get nodes
# Verification: You should see your Minikube node in a 'Ready' status.
```

```
# Check system pods in the kube-system namespace
kubectl get pods -n kube-system
# Verification: All pods in the 'kube-system' namespace should be in a 'Running' state.
```

Option B: EKS Setup (Production Environment)

If you're aiming for a production deployment, EKS is a robust choice.

Step 1.1: Install eksctl

eksctl is a simple CLI tool for creating and managing EKS clusters.

```
# Download and install eksctl
curl --silent --location
"https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_$(uname -
s)_amd64.tar.gz" | tar xz -C /tmp
sudo mv /tmp/eksctl /usr/local/bin

# Verify installation
eksctl version
```

Step 1.2: Create EKS Cluster Configuration

We'll define our EKS cluster's desired state in a YAML file. This includes the cluster name, region, and details about the worker nodes.

```
# Create cluster configuration file
cat > ~/kubernetes-microservices/cluster-config.yaml << EOF

apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig
metadata:
  name: microservices-cluster
  region: us-west-2 # Choose your desired AWS region
nodeGroups:
  - name: worker-nodes
    instanceType: t3.medium # Choose an appropriate instance type
    desiredCapacity: 3      # Number of worker nodes to start with
    minSize: 2             # Minimum number of worker nodes
    maxSize: 6             # Maximum number of worker nodes (for autoscaling)
    volumeSize: 20         # EBS volume size for each node (in GB)
    ssh:
      allow: true          # Allow SSH access to nodes (useful for debugging)
    iam:
      withAddonPolicies:
        autoScaler: true  # Grant permissions for Cluster Autoscaler
        cloudWatch: true  # Grant permissions for CloudWatch logs
        ebs: true         # Grant permissions for EBS volumes
cloudWatch:
```



```
clusterLogging:
  enable: ["all"]      # Enable all cluster logs to CloudWatch
EOF
```

Step 1.3: Deploy EKS Cluster

This step will create your EKS cluster based on the configuration file. It can take a significant amount of time.

```
# Create the cluster (this takes 15-20 minutes)
eksctl create cluster -f ~/kubernetes-microservices/cluster-config.yaml
# Verification: The command will output "EKS cluster 'microservices-cluster' in 'us-west-2' region is ready" upon success.
```

```
# Configure kubectl to use the new cluster
# This updates your kubeconfig file so kubectl commands target your EKS cluster.
aws eks update-kubeconfig --region us-west-2 --name microservices-cluster
```

```
# Verify cluster nodes are running
kubectl get nodes
# Verification: You should see your worker nodes in a 'Ready' status.
```

Note on EKS Ingress: For EKS, if you want to expose services externally via an AWS Application Load Balancer (ALB), you will typically need to install the AWS Load Balancer Controller. This guide focuses on internal ingress for simplicity, but for public access, you would integrate the ALB Controller and create Ingress resources with external DNS. You can find official documentation on installing the AWS Load Balancer Controller in the AWS EKS documentation.

Phase 2: Project Foundation Setup (30 minutes)

Now that your Kubernetes cluster is ready, let's set up the basic organizational components: namespaces and Role-Based Access Control (RBAC).

Step 2.1: Create Namespaces

Namespaces help you organize resources within your cluster. We'll create separate namespaces for our microservices, monitoring tools, and backup solution.

```
# Create namespaces configuration file
cat > ~/kubernetes-microservices/k8s/namespaces/namespaces.yaml << EOF
apiVersion: v1
kind: Namespace
metadata:
  name: microservices
```

```

labels:
  name: microservices
  purpose: application-workloads
---
apiVersion: v1
kind: Namespace
metadata:
  name: monitoring
  labels:
    name: monitoring
    purpose: observability
---
apiVersion: v1
kind: Namespace
metadata:
  name: velero
  labels:
    name: velero
    purpose: backup-restore
EOF

# Apply namespaces to your cluster
kubectl apply -f ~/kubernetes-microservices/k8s/namespaces/namespaces.yaml

# Verify namespaces were created
kubectl get namespaces
# Verification: You should see 'microservices', 'monitoring', and 'velero' listed in the
output.
```

Step 2.2: Create RBAC (Role-Based Access Control)

RBAC allows you to define permissions for users and services within your cluster. We'll create a ServiceAccount and a ClusterRole with ClusterRoleBinding for our microservices. This ensures they have only the necessary permissions.

```

# Create service account and RBAC configuration
cat > ~/kubernetes-microservices/k8s/rbac/service-account.yaml << EOF
apiVersion: v1
kind: ServiceAccount
metadata:
  name: microservices-sa
  namespace: microservices
  labels:
    app: microservices
---
apiVersion: rbac.authorization.k8s.io/v1
```

```

kind: ClusterRole
metadata:
  name: microservices-role
rules:
- apiGroups: [""] # "" indicates the core API group
  resources: ["pods", "services", "endpoints", "configmaps"]
  verbs: ["get", "list", "watch"] # Allow reading these resources
- apiGroups: ["apps"]
  resources: ["deployments", "replicasets"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["extensions", "networking.k8s.io"] # For Ingress resources
  resources: ["ingresses"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: microservices-binding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: microservices-role
subjects:
- kind: ServiceAccount
  name: microservices-sa
  namespace: microservices
EOF

# Apply RBAC configuration
kubectl apply -f ~/kubernetes-microservices/k8s/rbac/service-account.yaml

# Verify service account was created
kubectl get serviceaccount -n microservices
# Verification: You should see 'microservices-sa' listed in the 'microservices'
namespace.

```

Phase 3: Sample Microservices with Helm (45 minutes)

We'll create two simple Nginx-based microservices (Microservice A and Microservice B) and package them using Helm charts for easy deployment and management.

Step 3.1: Create Microservice A Helm Chart

3.1.1 Initialize Helm Chart

We'll start by creating a basic Helm chart structure for microservice-a.

```
# Navigate to helm directory
cd ~/kubernetes-microservices/helm

# Create helm chart structure for microservice-a
helm create microservice-a

# Remove default files we'll replace with our custom ones
rm microservice-a/templates/tests/test-connection.yaml
rm microservice-a/templates/hpa.yaml
rm microservice-a/templates/NOTES.txt
```

3.1.2 Update Chart Metadata

Edit the Chart.yaml file to include details specific to our project.

```
# Update Chart.yaml with our project details
cat > ~/kubernetes-microservices/helm/microservice-a/Chart.yaml << EOF
apiVersion: v2
name: microservice-a
description: Sample microservice A for demonstration
type: application
version: 0.1.0
appVersion: "1.21" # The version of the application (Nginx)
keywords:
  - microservice
  - nginx
  - demo
home: https://github.com/yourusername/kubernetes-microservices # Replace with your
GitHub URL
sources:
  - https://github.com/yourusername/kubernetes-microservices # Replace with your
GitHub URL
maintainers:
  - name: Your Name # Replace with your name
    email: your.email@example.com # Replace with your email
EOF
```

3.1.3 Configure Values File

The values.yaml file holds configurable parameters for our Helm chart. This is where we define image, replicas, resource requests/limits, ingress rules, and monitoring settings.

```
# Create comprehensive values.yaml
cat > ~/kubernetes-microservices/helm/microservice-a/values.yaml << EOF
replicaCount: 3 # Default number of replicas for the deployment
image:
```

```
repository: nginx
pullPolicy: IfNotPresent
tag: "1.21" # Nginx image tag
nameOverride: ""
fullnameOverride: ""
serviceAccount:
  create: false # We already created the service account manually
  name: "microservices-sa" # Use the service account we created
podAnnotations:
  prometheus.io/scrape: "true" # Instruct Prometheus to scrape metrics from this pod
  prometheus.io/port: "9113" # Port where the exporter exposes metrics
  prometheus.io/path: "/metrics" # Path where the exporter exposes metrics
podSecurityContext: # Security context for the pod
  runAsNonRoot: true
  runAsUser: 1000
  fsGroup: 2000
securityContext: # Security context for the container
  capabilities:
    drop:
      - ALL # Drop all Linux capabilities for security
  readOnlyRootFilesystem: false # Allow writing to certain paths (e.g., Nginx logs)
  allowPrivilegeEscalation: false
service:
  type: ClusterIP # Internal service, not directly exposed outside the cluster
  port: 80
  targetPort: 80
ingress:
  enabled: true
  className: "nginx"
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: / # Rewrite target for Nginx ingress
  hosts:
    - host: microservice-a.local # Custom hostname for accessing the service
      paths:
        - path: /
          pathType: Prefix
  tls: [] # No TLS configured by default; you'd add cert-manager here for production
resources: # Resource requests and limits for the main Nginx container
  limits:
    cpu: 500m # 0.5 CPU core
    memory: 512Mi
  requests:
    cpu: 250m # 0.25 CPU core
    memory: 256Mi
livenessProbe: # Checks if the container is running and healthy
  httpGet:
```

```
    path: /health # Custom health endpoint
    port: http
    initialDelaySeconds: 30
    periodSeconds: 10
readinessProbe: # Checks if the container is ready to serve traffic
  httpGet:
    path: /health
    port: http
    initialDelaySeconds: 5
    periodSeconds: 5
autoscaling: # HPA configuration
  enabled: true
  minReplicas: 2
  maxReplicas: 10
  targetCPUUtilizationPercentage: 70
  targetMemoryUtilizationPercentage: 80
nodeSelector: {}
tolerations: []
affinity: {}
monitoring:
  enabled: true
  serviceMonitor: # Configuration for Prometheus ServiceMonitor
    enabled: true
    interval: 30s
    path: /metrics
    port: metrics
nginxConfig: # Custom Nginx configuration
  enabled: true
  data: |
    server {
      listen 80;
      server_name localhost;

      location / {
        root /usr/share/nginx/html;
        index index.html index.htm;
      }

      location /health {
        access_log off;
        return 200 "healthy\n";
        add_header Content-Type text/plain;
      }

      location /nginx_status {
        stub_status on;
```

```

        access_log off;
        allow 127.0.0.1; # Only allow access from localhost for security
        deny all;
    }
}
EOF

```

3.1.4 Create Enhanced Deployment Template

This template defines how our microservice pods will be deployed, including containers, resource limits, probes, and volume mounts for the Nginx configuration. It also includes an Nginx Prometheus exporter sidecar.

```

# Create detailed deployment template
cat > ~/kubernetes-microservices/helm/microservice-a/templates/deployment.yaml <<
EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "microservice-a.fullname" . }}
  namespace: microservices
  labels:
    {{- include "microservice-a.labels" . | nindent 4 }}
spec:
  {{- if not .Values.autoscaling.enabled }}
  replicas: {{ .Values.replicaCount }}
  {{- end }}
  selector:
    matchLabels:
      {{- include "microservice-a.selectorLabels" . | nindent 6 }}
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  template:
    metadata:
      annotations:
        checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . |
sha256sum }}
        {{- with .Values.podAnnotations }}
        {{- toYaml . | nindent 8 }}
        {{- end }}
      labels:
        {{- include "microservice-a.selectorLabels" . | nindent 8 }}
        version: {{ .Values.image.tag }}

```

```

spec:
  {{- with .Values.imagePullSecrets }}
  imagePullSecrets:
    {{- toYaml . | nindent 8 }}
  {{- end }}
  serviceAccountName: {{ .Values.serviceAccount.name }}
  securityContext:
    {{- toYaml .Values.podSecurityContext | nindent 8 }}
  containers:
    - name: {{ .Chart.Name }}
      securityContext:
        {{- toYaml .Values.securityContext | nindent 12 }}
      image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default
.Chart.AppVersion }}"
      imagePullPolicy: {{ .Values.image.pullPolicy }}
      ports:
        - name: http
          containerPort: 80
          protocol: TCP
      {{- if .Values.livenessProbe }}
      livenessProbe:
        {{- toYaml .Values.livenessProbe | nindent 12 }}
      {{- end }}
      {{- if .Values.readinessProbe }}
      readinessProbe:
        {{- toYaml .Values.readinessProbe | nindent 12 }}
      {{- end }}
      resources:
        {{- toYaml .Values.resources | nindent 12 }}
      {{- if .Values.nginxConfig.enabled }}
      volumeMounts:
        - name: nginx-config
          mountPath: /etc/nginx/conf.d/default.conf
          subPath: nginx.conf
          readOnly: true
      {{- end }}
    - name: nginx-exporter # Sidecar container for Prometheus metrics
      image: nginx/nginx-prometheus-exporter:0.11.0
      args:
        - -nginx.scrape-uri=http://localhost/nginx_status
      ports:
        - name: metrics
          containerPort: 9113
          protocol: TCP
      resources:
        requests:

```



```

      cpu: 10m
      memory: 32Mi
    limits:
      cpu: 50m
      memory: 64Mi
    livenessProbe:
      httpGet:
        path: /metrics
        port: metrics
      initialDelaySeconds: 30
      periodSeconds: 10
    readinessProbe:
      httpGet:
        path: /metrics
        port: metrics
      initialDelaySeconds: 5
      periodSeconds: 5
  {{- if .Values.nginxConfig.enabled }}
  volumes:
    - name: nginx-config
      configMap:
        name: {{ include "microservice-a.fullname" . }}-config
  {{- end }}
  {{- with .Values.nodeSelector }}
  nodeSelector:
    {{- toYaml . | nindent 8 }}
  {{- end }}
  {{- with .Values.affinity }}
  affinity:
    {{- toYaml . | nindent 8 }}
  {{- end }}
  {{- with .Values.tolerations }}
  tolerations:
    {{- toYaml . | nindent 8 }}
  {{- end }}
EOF

```

3.1.5 Create ConfigMap Template

This ConfigMap will hold our custom Nginx configuration, which is then mounted into the Nginx container.

```

# Create ConfigMap for nginx configuration
cat > ~/kubernetes-microservices/helm/microservice-a/templates/configmap.yaml <<
EOF
{{- if .Values.nginxConfig.enabled }}

```

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ include "microservice-a.fullname" . }}-config
  namespace: microservices
  labels:
    {{- include "microservice-a.labels" . | nindent 4 }}
data:
  nginx.conf: |
{{ .Values.nginxConfig.data | indent 4 }}
{{- end }}
EOF

```

3.1.6 Update Service Template

This service exposes our Nginx microservice within the cluster and also exposes the Prometheus exporter port.

```

# Create enhanced service template
cat > ~/kubernetes-microservices/helm/microservice-a/templates/service.yaml << EOF
apiVersion: v1
kind: Service
metadata:
  name: {{ include "microservice-a.fullname" . }}
  namespace: microservices
  labels:
    {{- include "microservice-a.labels" . | nindent 4 }}
  {{- if .Values.monitoring.enabled }}
  annotations:
    prometheus.io/scrape: "true"
    prometheus.io/port: "9113"
    prometheus.io/path: "/metrics"
  {{- end }}
spec:
  type: {{ .Values.service.type }}
  ports:
    - port: {{ .Values.service.port }}
      targetPort: http
      protocol: TCP
      name: http
    {{- if .Values.monitoring.enabled }}
    - port: 9113
      targetPort: metrics
      protocol: TCP
      name: metrics
    {{- end }}

```

```
selector:
  {{- include "microservice-a.selectorLabels" . | nindent 4 }}
EOF
```

Step 3.2: Create Microservice B

Instead of recreating everything, we'll copy Microservice A's chart and modify it for Microservice B.

```
# Copy microservice-a to create microservice-b
cp -r ~/kubernetes-microservices/helm/microservice-a ~/kubernetes-
microservices/helm/microservice-b

# Update Chart.yaml for microservice-b
sed -i 's/microservice-a/microservice-b/g' ~/kubernetes-
microservices/helm/microservice-b/Chart.yaml
sed -i 's/Sample microservice A/Sample microservice B/g' ~/kubernetes-
microservices/helm/microservice-b/Chart.yaml

# Update values.yaml for microservice-b (change ingress host)
sed -i 's/microservice-a\.local/microservice-b.local/g' ~/kubernetes-
microservices/helm/microservice-b/values.yaml

# Update all template files to use microservice-b naming
find ~/kubernetes-microservices/helm/microservice-b/templates -name "*.yaml" -exec
sed -i 's/microservice-a/microservice-b/g' {} \;
```

Step 3.3: Deploy Microservices

Now, let's deploy both microservices using Helm.

```
# Navigate back to the main project directory
cd ~/kubernetes-microservices

# Install microservice-a using Helm
helm install microservice-a ~/kubernetes-microservices/helm/microservice-a \
  --namespace microservices \
  --wait \
  --timeout=300s

# Install microservice-b using Helm
helm install microservice-b ~/kubernetes-microservices/helm/microservice-b \
  --namespace microservices \
  --wait \
  --timeout=300s
```

```
# Check deployment status
kubectl get pods -n microservices
# Verification: You should see 3 pods for microservice-a and 3 pods for microservice-b,
all in 'Running' status.
kubectl get svc -n microservices
kubectl get ingress -n microservices
helm list -n microservices
```

Accessing the Microservices (Minikube):

For Minikube, you'll need to update your /etc/hosts file (or C:\Windows\System32\drivers\etc\hosts on Windows) to map the microservice-a.local and microservice-b.local hostnames to your Minikube IP.

1. Get your Minikube IP:
`minikube ip`
2. Add entries to your hosts file:
`<MINIKUBE_IP> microservice-a.local`
`<MINIKUBE_IP> microservice-b.local`

(Replace <MINIKUBE_IP> with the actual IP from the previous command).

Now, you can access your microservices in your web browser:

- `http://microservice-a.local`
- `http://microservice-b.local`

Phase 4: Auto-scaling Setup (30 minutes)

Kubernetes offers powerful auto-scaling capabilities. We'll set up both Horizontal Pod Autoscaler (HPA) and Vertical Pod Autoscaler (VPA).

Step 4.1: Horizontal Pod Autoscaler (HPA)

HPA automatically scales the number of pod replicas in a deployment based on observed CPU utilization or other select metrics. We already enabled metrics-server in Minikube, which is a prerequisite. For EKS, metrics-server is usually installed by default or can be easily added.

```
# Navigate to the k8s/autoscaling directory
cd ~/kubernetes-microservices/k8s/autoscaling
```

```
# Create HPA configuration for both microservices
cat > hpa.yaml << EOF
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
```

```

metadata:
  name: microservice-a-hpa
  namespace: microservices
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: microservice-a
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70 # Target 70% CPU utilization
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80 # Target 80% memory utilization
  behavior: # Define scaling behavior
    scaleDown:
      stabilizationWindowSeconds: 300 # Wait 5 minutes before scaling down
      policies:
        - type: Percent
          value: 10 # Scale down by 10% of current replicas
          periodSeconds: 60
    scaleUp:
      stabilizationWindowSeconds: 60 # Wait 1 minute before scaling up
      policies:
        - type: Percent
          value: 50 # Scale up by 50% of current replicas
          periodSeconds: 15
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: microservice-b-hpa
  namespace: microservices
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment

```

```
    name: microservice-b
minReplicas: 2
maxReplicas: 10
metrics:
- type: Resource
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 70
- type: Resource
  resource:
    name: memory
    target:
      type: Utilization
      averageUtilization: 80
behavior:
  scaleDown:
    stabilizationWindowSeconds: 300
    policies:
    - type: Percent
      value: 10
      periodSeconds: 60
  scaleUp:
    stabilizationWindowSeconds: 60
    policies:
    - type: Percent
      value: 50
      periodSeconds: 15
EOF
```

```
# Apply HPA configuration
kubectl apply -f hpa.yaml
```

```
# Verify HPA is working (it might take a minute or two for metrics to show)
kubectl get hpa -n microservices
# Verification: You should see the HPA resources listed. The 'TARGETS' column might
show '<unknown>' initially, but should populate with CPU/Memory utilization
percentages after a few minutes.
```

You can generate some load on your microservices (e.g., using hey or apachebench) to see the HPA in action.

Step 4.2: Vertical Pod Autoscaler (VPA)

VPA automatically adjusts the CPU and memory requests and limits for containers based on their historical usage. This helps optimize resource allocation and prevent resource contention.

Important Note on VPA and HPA Interaction: VPA and HPA can sometimes conflict if both are trying to manage the same resource (e.g., CPU). VPA adjusts the *resource requests* of a pod, which directly influences the *utilization percentage* that HPA uses for its scaling decisions. For this guide, we'll set updateMode: "Off" for VPA to only get recommendations. This is a safer approach when using both HPA and VPA, allowing HPA to manage horizontal scaling based on utilization, while VPA provides insights into optimal resource requests without directly modifying them. In a production scenario, you would carefully consider which autoscaler manages which resource, or use a more advanced VPA mode with careful testing.

```
# Install VPA components (this clones the Kubernetes autoscaler repo and runs an install script)
```

```
cd /tmp
```

```
git clone https://github.com/kubernetes/autoscaler.git
```

```
cd autoscaler/vertical-pod-autoscaler/
```

```
./hack/vpa-install.sh
```

```
# Navigate back to the k8s/autoscaling directory
```

```
cd ~/kubernetes-microservices/k8s/autoscaling
```

```
# Create VPA configuration for both microservices
```

```
cat > vpa.yaml << EOF
```

```
apiVersion: autoscaling.k8s.io/v1
```

```
kind: VerticalPodAutoscaler
```

```
metadata:
```

```
  name: microservice-a-vpa
```

```
  namespace: microservices
```

```
spec:
```

```
  targetRef:
```

```
    apiVersion: apps/v1
```

```
    kind: Deployment
```

```
    name: microservice-a
```

```
  updatePolicy:
```

```
    updateMode: "Off" # Start with "Off" to see recommendations only
```

```
  resourcePolicy:
```

```
    containerPolicies:
```

```
      - containerName: microservice-a
```

```
        maxAllowed:
```

```

        cpu: 1 # Max 1 CPU core
        memory: 2Gi # Max 2GB memory
    minAllowed:
        cpu: 100m # Min 0.1 CPU core
        memory: 128Mi # Min 128MB memory
    controlledResources: ["cpu", "memory"]
---
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: microservice-b-vpa
  namespace: microservices
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: microservice-b
  updatePolicy:
    updateMode: "Off"
  resourcePolicy:
    containerPolicies:
      - containerName: microservice-b
        maxAllowed:
          cpu: 1
          memory: 2Gi
        minAllowed:
          cpu: 100m
          memory: 128Mi
        controlledResources: ["cpu", "memory"]
EOF

# Apply VPA configuration
kubectl apply -f vpa.yaml

# To see VPA recommendations (it takes some time for VPA to gather data):
# kubectl describe vpa microservice-a-vpa -n microservices
# Verification: Look for the 'Recommended Resources' section in the output of 'kubectl
describe vpa'.

```


Phase 5: Monitoring Stack Setup (60 minutes)

A robust monitoring stack is crucial for understanding the health and performance of your microservices. We'll use Prometheus for data collection and Grafana for visualization.

Step 5.1: Install Prometheus with Helm

We'll use the kube-prometheus-stack Helm chart, which bundles Prometheus, Grafana, Alertmanager, and various exporters.

```
# Navigate to the monitoring/prometheus directory
cd ~/kubernetes-microservices/monitoring/prometheus
```

```
# Add Prometheus Helm repository
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

```
# Create custom values for Prometheus stack
cat > values.yaml << EOF
prometheus:
  prometheusSpec:
    retention: 7d      # Keep metrics for 7 days
    retentionSize: "10GB" # Max storage size for metrics
    storageSpec:
      volumeClaimTemplate:
        spec:
          storageClassName: standard # Use 'standard' storage class (default for most
clusters)
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 20Gi # Request 20GB of persistent storage for Prometheus
    serviceMonitorSelectorNilUsesHelmValues: false
    podMonitorSelectorNilUsesHelmValues: false
    ruleSelectorNilUsesHelmValues: false
    serviceMonitorNamespaceSelector: {}
    serviceMonitorSelector: {}
    podMonitorNamespaceSelector: {}
    podMonitorSelector: {}
    ruleNamespaceSelector: {}
    ruleSelector: {}
grafana:
  enabled: true
  # It's recommended to store sensitive data like admin passwords in Kubernetes
Secrets
  # and reference them here using a template function or by manually creating the
```

secret.

For this guide, we'll keep it direct for simplicity.

adminPassword: "admin123" # Set a strong password for Grafana admin user

ingress:

enabled: true

hosts:

- grafana.local # Custom hostname for Grafana UI

persistence:

enabled: true

size: 10Gi # Request 10GB of persistent storage for Grafana data

alertmanager:

enabled: true

ingress:

enabled: true

hosts:

- alertmanager.local # Custom hostname for Alertmanager UI

kubeStateMetrics:

enabled: true # Collects metrics about Kubernetes objects

nodeExporter:

enabled: true # Collects metrics about cluster nodes

prometheusOperator:

enabled: true # Manages Prometheus and Alertmanager instances

defaultRules:

create: true # Create default Prometheus alerting rules

EOF

Install Prometheus stack using Helm

helm install prometheus-stack prometheus-community/kube-prometheus-stack \

--namespace monitoring \

--values values.yaml \

--wait \

--timeout=600s # This can take a while

Verify installation

kubectl get pods -n monitoring

Verification: You should see several pods running, including 'prometheus-stack-kube-prom-prometheus-0', 'prometheus-stack-grafana-...', 'prometheus-stack-kube-state-metrics-...', and 'prometheus-stack-kube-prom-alertmanager-0'. All should be in a 'Running' state.

kubectl get svc -n monitoring

Step 5.2: Create ServiceMonitor for Microservices

For Prometheus to scrape metrics from our custom microservices, we need to define ServiceMonitor resources. These tell Prometheus where to find the metrics endpoints.

```
# Create ServiceMonitor configuration
cat > servicemonitor.yaml << EOF
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: microservices-a-monitor
  namespace: monitoring # ServiceMonitor should be in the same namespace as
Prometheus
labels:
  app: microservices
  release: prometheus-stack # This label helps Prometheus discover the
ServiceMonitor
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: microservice-a # Selects the service for microservice-a
  namespaceSelector:
    matchNames:
      - microservices # Scrape services in the 'microservices' namespace
  endpoints:
    - port: metrics # This corresponds to the 'metrics' port defined in our microservice
service
    interval: 30s
    path: /metrics
---
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: microservices-b-monitor
  namespace: monitoring
labels:
  app: microservices
  release: prometheus-stack
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: microservice-b
  namespaceSelector:
    matchNames:
      - microservices
  endpoints:
    - port: metrics
```

```
    interval: 30s
    path: /metrics
EOF
```

```
# Apply ServiceMonitor
kubectl apply -f servicemonitor.yaml
# Verification: You can check if Prometheus has picked up the targets by port-
forwarding Prometheus UI:
# kubectl -n monitoring port-forward svc/prometheus-stack-kube-prom-prometheus
9090:9090
# Then open http://localhost:9090/targets in your browser. You should see
'microservice-a' and 'microservice-b' targets listed.
```

Step 5.3: Create Custom Grafana Dashboard

We'll create a custom Grafana dashboard to visualize key metrics of our microservices. This dashboard will be deployed as a ConfigMap and automatically picked up by Grafana.

```
# Navigate to the monitoring/grafana directory
cd ~/kubernetes-microservices/monitoring/grafana

# Create custom dashboard ConfigMap
cat > dashboard-configmap.yaml << EOF
apiVersion: v1
kind: ConfigMap
metadata:
  name: microservices-dashboard
  namespace: monitoring
  labels:
    grafana_dashboard: "1" # This label tells Grafana to import this ConfigMap as a
dashboard
data:
  microservices-dashboard.json: |
    {
      "dashboard": {
        "id": null,
        "title": "Microservices Dashboard",
        "tags": ["kubernetes", "microservices"],
        "style": "dark",
        "timezone": "browser",
        "time": {
          "from": "now-1h",
          "to": "now"
        },
      },
      "refresh": "10s",
```

```

"panels": [
  {
    "id": 1,
    "title": "Pod Status",
    "type": "stat",
    "targets": [
      {
        "expr": "kube_pod_status_ready{namespace=\"microservices\"}",
        "legendFormat": "{{pod}}"
      }
    ],
    "fieldConfig": {
      "defaults": {
        "color": {"mode": "thresholds"},
        "thresholds": {
          "steps": [
            {"color": "red", "value": 0},
            {"color": "green", "value": 1}
          ]
        }
      }
    },
    "gridPos": {"h": 8, "w": 12, "x": 0, "y": 0}
  },
  {
    "id": 2,
    "title": "CPU Usage",
    "type": "graph",
    "targets": [
      {
        "expr":
"rate(container_cpu_usage_seconds_total{namespace=\"microservices\",container!=\"POD\",container!=\"\"}[5m]) * 100",
        "legendFormat": "{{pod}}"
      }
    ],
    "gridPos": {"h": 8, "w": 12, "x": 12, "y": 0}
  },
  {
    "id": 3,
    "title": "Memory Usage",
    "type": "graph",
    "targets": [
      {
        "expr":
"container_memory_usage_bytes{namespace=\"microservices\",container!=\"POD\",con

```

```

tainer!="\"} / (1024 * 1024)",
    "legendFormat": "{{pod}}"
  }
],
"gridPos": {"h": 8, "w": 12, "x": 0, "y": 8}
},
{
  "id": 4,
  "title": "HTTP Request Rate",
  "type": "graph",
  "targets": [
    {
      "expr": "sum(rate(nginx_http_requests_total{job=\"microservice-a\"}[5m]))",
      "legendFormat": "Microservice A Requests"
    },
    {
      "expr": "sum(rate(nginx_http_requests_total{job=\"microservice-b\"}[5m]))",
      "legendFormat": "Microservice B Requests"
    }
  ],
  "gridPos": {"h": 8, "w": 12, "x": 12, "y": 8}
}
]
}
}
EOF

```

```

# Apply custom dashboard
kubectl apply -f dashboard-configmap.yaml

```

Step 5.4: Access Grafana Dashboard

To access the Grafana dashboard, you'll first need to get the ingress URL.

- For Minikube:
Get the ingress URL for Grafana:
`kubectl get ingress -n monitoring`

Look for the grafana.local host in the output. You might need to add grafana.local to your /etc/hosts file (or C:\Windows\System32\drivers\etc\hosts on Windows) mapping it to the Minikube IP address. Get the Minikube IP with minikube ip.
Example /etc/hosts entry:
`<MINIKUBE_IP> grafana.local`

Once configured, open your web browser and navigate to `http://grafana.local`.

- For EKS:
The Grafana ingress will have an external IP or hostname assigned by AWS Load Balancer.
`kubectl get ingress -n monitoring -o wide`

Copy the ADDRESS value for the Grafana ingress and paste it into your web browser.

Login Credentials:

The default Grafana username is admin.

The password you set in the values.yaml for Prometheus stack was admin123.

Once logged in, navigate to **Dashboards** -> **Manage** and you should see the "Microservices Dashboard" listed. Click on it to view your custom dashboard with metrics for your microservices.

Phase 6: Backup Solution with Velero (45 minutes)

Velero is an open-source tool to safely backup and restore, perform disaster recovery, and migrate Kubernetes cluster resources and persistent volumes.

Step 6.1: Install Velero CLI

First, download and install the Velero command-line interface (CLI).

Download the latest Velero release

`VELERO_VERSION="v1.12.0" # You can check for the latest stable version on Velero's GitHub releases page`

`curl -LO https://github.com/vmware-`

`tanzu/velero/releases/download/${VELERO_VERSION}/velero-${VELERO_VERSION}-linux-amd64.tar.gz`

Extract the tarball

`tar -xvf velero-${VELERO_VERSION}-linux-amd64.tar.gz`

Move the Velero binary to your system's PATH

`sudo mv velero-${VELERO_VERSION}-linux-amd64/velero /usr/local/bin/`

Verify installation

`velero version`

Step 6.2: Configure Velero with AWS S3 (for EKS) or Minio (for Minikube)

Velero needs an object storage location to store backups.

Option A: For EKS (AWS S3)

1. **Create an S3 Bucket:** You'll need an S3 bucket in your AWS account to store Velero backups. Choose a unique name.

```
aws s3 mb s3://<YOUR_VELERO_BUCKET_NAME> --region us-west-2
# Replace with your desired region
```

Replace <YOUR_VELERO_BUCKET_NAME> with a unique S3 bucket name.

2. **Create an IAM User and Policy:** Velero requires IAM permissions to access the S3 bucket. Create an IAM policy and user, and attach the policy to the user. Create a file named velero-policy.json in your ~/kubernetes-microservices/backup directory:

```
# Navigate to the backup directory
cd ~/kubernetes-microservices/backup
```

```
cat > velero-policy.json << EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeVolumes",
        "ec2:DescribeSnapshots",
        "ec2:CreateSnapshot",
        "ec2>DeleteSnapshot",
        "ec2:DescribeVolumeStatus"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3>DeleteObject",
        "s3:PutObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::<YOUR_VELERO_BUCKET_NAME>/*",
        "arn:aws:s3:::<YOUR_VELERO_BUCKET_NAME>"
      ]
    }
  ]
}
```



```
}
  ]
}
]
```

EOF

`` Replace <YOUR_VELERO_BUCKET_NAME> with your S3 bucket name in the JSON content.

Apply the policy and create the user:

```
``bash
aws iam create-policy --policy-name VeleroBackupPolicy --policy-document file://velero-policy.json
# IMPORTANT: Copy the ARN (Amazon Resource Name) of the created policy from the output.
aws iam create-user --user-name velero-user
aws iam attach-user-policy --user-name velero-user --policy-arn
arn:aws:iam::<YOUR_AWS_ACCOUNT_ID>:policy/VeleroBackupPolicy # Replace with
your AWS Account ID and copied policy ARN
aws iam create-access-key --user-name velero-user > velero-credentials.txt
``
```

Make sure to replace ``<YOUR_AWS_ACCOUNT_ID>`` with your actual AWS account ID and the policy ARN.

****Best Practice:**** Instead of using a local file for credentials, it's more secure to create a Kubernetes secret and reference it during Velero installation.

```
``bash
# Create Kubernetes secret for AWS credentials
kubectl create secret generic velero-aws-credentials \
  --namespace velero \
  --from-file=cloud=./credentials-velero # This file contains your AWS_ACCESS_KEY_ID
and AWS_SECRET_ACCESS_KEY
``
```

3. Install Velero (EKS):

```
velero install \
--provider aws \
--plugins velero/aws:v1.12.0 \
--bucket <YOUR_VELERO_BUCKET_NAME> \
--secret-key-selector cloud=velero-aws-credentials \
--backup-location-config region=us-west-2 \
--snapshot-location-config region=us-west-2 \
--namespace velero
```

Replace <YOUR_VELERO_BUCKET_NAME> with your S3 bucket name.

Option B: For Minikube (Minio)

For local development with Minikube, Velero can use Minio as an S3-compatible object storage.

1. Deploy Minio on Minikube:

```
# Create the velero namespace if you haven't already
kubectl create namespace velero
```

```
helm repo add minio https://helm.min.io/
helm install minio minio/minio \
--namespace velero \
--set accessKey=minio \
--set secretKey=minio123 \
--set persistence.size=10Gi
```

2. Get Minio Service IP/Port:

```
kubectl get svc -n velero minio
```

Note the CLUSTER-IP and PORT for the Minio service. These will be used in the Velero installation command.

3. Create Minio Credentials File:

Create a file named credentials-velero in your ~/kubernetes-microservices/backup directory with the following content:

```
# Navigate to the backup directory
cd ~/kubernetes-microservices/backup

cat > credentials-velero << EOF
[default]
aws_access_key_id=minio
aws_secret_access_key=minio123
```

EOF ``

4. Install Velero (Minikube with Minio):

```
velero install \
--provider aws \
--plugins velero/aws:v1.12.0 \
--bucket velero \
--secret-file ./credentials-velero \
--use-node-agent \
--namespace velero \
--backup-location-config
region=minio,s3ForcePathStyle=true,s3Url=http://<MINIO_CLUSTER_IP>:<MINIO
_PORT>
```

Replace <MINIO_CLUSTER_IP> and <MINIO_PORT> with the actual Cluster IP and Port of your Minio service obtained earlier.

Verify Velero Installation:

```
kubectl get pods -n velero
# Verification: You should see Velero pods (e.g., 'velero-...') and, if using Minio, 'minio-...' pods in a 'Running' state.
```

Step 6.3: Create a Backup

Let's create a backup of your microservices namespace.

```
# Navigate back to the main project directory
cd ~/kubernetes-microservices
```

```
# Create a backup of the microservices namespace
velero backup create microservices-backup --include-namespaces microservices
```

```
# Check the status of the backup
velero backup get
# Verification: The backup should eventually show 'Completed' in the 'PHASE' column.
```

```
# Get detailed information about the backup
velero backup describe microservices-backup
```

The Phase should eventually change to Completed.

Step 6.4: Restore from Backup

To test the restore functionality, let's simulate a disaster by deleting the microservices namespace and then restoring it.

Delete the microservices namespace (simulate disaster)

```
kubectl delete namespace microservices
```

Verify the namespace is gone

```
kubectl get namespaces
```

Verification: The 'microservices' namespace should no longer be listed.

Now, restore the namespace from your backup.

Restore from the backup

```
velero restore create --from-backup microservices-backup
```

Check the status of the restore (it might take a moment for pods to come up)

```
velero restore get
```

Verification: The restore should eventually show 'Completed' in the 'PHASE' column.

Get detailed information about the restore (the restore name will include a timestamp)

Replace <TIMESTAMP> with the actual restore name from 'velero restore get' output

```
velero restore describe microservices-backup-<TIMESTAMP>
```

After the restore completes, verify that your microservices pods are running again in the microservices namespace.

```
kubectl get pods -n microservices
```

Verification: Your microservice pods should be back in a 'Running' state.

```
kubectl get svc -n microservices
```

Phase 7: Alerting with Prometheus Alertmanager & Grafana (30 minutes)

This phase focuses on setting up alerts to notify you when certain conditions in your microservices platform are met.

Step 7.1: Configure Alertmanager

The Prometheus Helm chart you installed already includes Alertmanager. You can configure Alertmanager to send notifications to various receivers (email, Slack, PagerDuty, etc.). For this guide, we'll demonstrate a basic configuration.

First, let's get the Alertmanager service in the monitoring namespace:

```
kubectl get svc -n monitoring | grep alertmanager
```

You should see a service like prometheus-kube-prometheus-alertmanager.

You can access the Alertmanager UI by port-forwarding:

```
kubectl -n monitoring port-forward svc/prometheus-kube-prometheus-alertmanager 9093:9093
```

Then, open your browser to <http://localhost:9093>.

To configure Alertmanager, you typically modify its configuration through a Secret or by setting values in the Helm chart. For persistent configuration, it's best to update your monitoring/prometheus/values.yaml file.

Let's add a simple receiver for demonstration purposes (e.g., to log alerts). In a production environment, you would configure actual notification channels.

Navigate to the monitoring/prometheus directory:

```
cd ~/kubernetes-microservices/monitoring/prometheus
```

Edit values.yaml to add the alertmanager.config.alertmanager.yml section under alertmanager:

```
# Use sed to add the alertmanager config block directly into the values.yaml
# This command is a bit complex, but it ensures the content is added correctly.
# It finds the 'alertmanager:' line and inserts the config block after it.
sed -i '/^alertmanager:/a \
  config:\
    alertmanager.yml: |\
      global:\
        resolve_timeout: 5m\
      route:\
        group_by: ["alertname", "job"]\

```

```

    group_wait: 30s\
    group_interval: 5m\
    repeat_interval: 1h\
    receiver: "\"default-receiver\""\
receivers:\
  - name: "\"default-receiver\""\
    webhook_configs:\
      - url: "\"http://localhost:8080/webhook\"" # Placeholder - replace with actual
notification endpoint\
# You can add more receivers here, e.g., for Slack, email, PagerDuty\
# - name: "\"slack-receiver\""\
#   slack_configs:\
#     - channel: "\"#alerts\""\
#       api_url: "\"<YOUR_SLACK_WEBHOOK_URL>\""\
#       text: "\"{{ .CommonLabels.alertname }} fired!\""\
# - name: "\"email-receiver\""\
#   email_configs:\
#     - to: "\"your.email@example.com\""\
#       from: "\"alertmanager@example.com\""\
#       smarthost: "\"smtp.example.com:587\""\
#       auth_username: "\"user\""\
#       auth_password: "\"password\""
' values.yaml

```

Note: The sed command above is designed to insert the block. You can also manually open values.yaml and add the config block under alertmanager:.

After modifying the values.yaml, upgrade your Prometheus stack to apply the changes:

```

helm upgrade prometheus-stack prometheus-community/kube-prometheus-stack \
--namespace monitoring \
--values values.yaml \
--wait \
--timeout=600s

```

Step 7.2: Create Prometheus Alerting Rules

Prometheus uses alerting rules to identify alert conditions. These rules are typically defined in PrometheusRule objects when using kube-prometheus-stack.

Let's create a simple alerting rule that fires an alert if a microservice pod is not ready, and another for high CPU load.

```
# Create Prometheus alerting rules file
```

```
cat > alerts.yaml << EOF
```

```
apiVersion: monitoring.coreos.com/v1
```

```
kind: PrometheusRule
```

```
metadata:
```

```
  name: microservices-alerts
```

```
  namespace: monitoring
```

```
  labels:
```

```
    release: prometheus-stack # This label is important for the Prometheus operator to  
pick up the rule
```

```
spec:
```

```
  groups:
```

```
  - name: microservice-pod-alerts
```

```
    rules:
```

```
    - alert: MicroservicePodNotReady
```

```
      expr: kube_pod_status_ready{namespace="microservices"} == 0
```

```
      for: 1m # Alert if not ready for 1 minute
```

```
      labels:
```

```
        severity: critical
```

```
      annotations:
```

```
        summary: "Microservice pod {{ $labels.pod }} is not ready in namespace {{  
$labels.namespace }}"
```

```
        description: "The pod {{ $labels.pod }} in namespace {{ $labels.namespace }} has  
been in a not ready state for more than 1 minute."
```

```
    - alert: HighCPULoad
```

```
      expr: avg(rate(container_cpu_usage_seconds_total{namespace="microservices",  
container!=""}[5m])) by (pod) * 100 > 80
```

```
      for: 5m # Alert if CPU is high for 5 minutes
```

```
      labels:
```

```
        severity: warning
```

```
      annotations:
```

```
        summary: "High CPU usage on pod {{ $labels.pod }} in namespace {{  
$labels.namespace }}"
```

```
        description: "The pod {{ $labels.pod }} in namespace {{ $labels.namespace }} has  
been experiencing high CPU usage (over 80%) for 5 minutes."
```

```
EOF
```

```
# Apply the Prometheus alerting rules
```

```
kubectl apply -f alerts.yaml
```

Prometheus will automatically discover and load these rules. You can verify them by accessing the Prometheus UI (you might need to port-forward the Prometheus service similar to Alertmanager) and navigating to "Alerts."

Step 7.3: Configure Grafana Notifications

Grafana can be configured to send notifications based on alerts generated by Prometheus. You can configure notification channels within Grafana's UI (Configuration -> Notification channels).

Common notification channels include:

- **Email:** Send alerts to specified email addresses.
- **Slack:** Post alerts to a Slack channel.
- **Webhook:** Send alerts to a custom webhook endpoint.

To access Grafana and configure notification channels:

1. Open Grafana UI (as in Step 5.4).
2. Go to Alerting -> Notification channels.
3. Click Add channel.
4. Choose your desired type (e.g., Slack, Email) and fill in the necessary details.

Step 7.4: Test Alerts

To test your alerts, you can intentionally create a condition that triggers them.

Testing MicroservicePodNotReady Alert:

You can scale down one of your microservices to zero replicas and then scale it back up to trigger the MicroservicePodNotReady alert.

Scale down microservice-a deployment to 0 replicas

```
kubectl scale deployment microservice-a -n microservices --replicas=0
```

Wait for a few minutes (e.g., 2-3 minutes) for the alert to fire

You can check Alertmanager UI (<http://localhost:9093>) or your configured notification channel.

Scale microservice-a back up to its original replica count (e.g., 3)

Navigate back to the main project directory first

```
cd ~/kubernetes-microservices
```

```
helm upgrade microservice-a ~/kubernetes-microservices/helm/microservice-a \
  --namespace microservices \
  --set replicaCount=3 \
  --wait
```


You should observe an alert in your configured notification channel (if set up) or in the Alertmanager UI. The alert should resolve automatically once the pod becomes ready again.

Testing HighCPULoad Alert:

To simulate high CPU load, you can run a temporary busybox pod that generates CPU load against one of your microservices.

```
kubectl run -it --rm --restart=Never busybox --image=busybox --namespace
microservices -- /bin/sh
# Inside the busybox pod, run this command to generate continuous HTTP requests:
# while true; do wget -q -O- http://microservice-a.microservices.svc.cluster.local; done
# Let it run for a few minutes to generate load.
# Then exit the busybox pod by typing 'exit'.
```

After a few minutes, check your Alertmanager or notification channel for the HighCPULoad alert.

Phase 7.5 (Optional): Securing Communication with Network Policies

Network Policies allow you to control traffic flow between pods in your Kubernetes cluster. This is a crucial security measure to ensure that only authorized connections are allowed.

Let's create a simple Network Policy that restricts traffic to our microservices.

```
# Navigate to the k8s/network directory
cd ~/kubernetes-microservices/k8s/network

# Create Network Policy configuration
cat > microservices-network-policy.yaml << EOF
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-ingress-to-microservices
  namespace: microservices
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: microservice-a # Apply to microservice-a pods
  policyTypes:
    - Ingress
  ingress:
```

```

- from:
  - namespaceSelector:
      matchLabels:
        name: ingress-nginx # Allow traffic from the ingress-nginx controller
namespace
  podSelector:
      matchLabels:
        app.kubernetes.io/name: ingress-nginx # Select ingress-nginx pods
  ports:
    - protocol: TCP
      port: 80 # Allow traffic to port 80 of microservice-a
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: restrict-microservice-b-access
  namespace: microservices
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: microservice-b # Apply to microservice-b pods
  policyTypes:
    - Ingress
  ingress:
    # This policy explicitly denies all ingress traffic to microservice-b
    # unless explicitly allowed by another rule.
    # For demonstration, we'll allow traffic only from microservice-a (if needed for inter-
service communication)
    - from:
        - podSelector:
            matchLabels:
              app.kubernetes.io/name: microservice-a # Allow traffic from microservice-a
      ports:
        - protocol: TCP
          port: 80
EOF

```

Apply Network Policies

```
kubectl apply -f microservices-network-policy.yaml
```

Verification: You can test these policies by trying to curl microservice-b from a pod in a different namespace (it should fail).

```
# kubectl run -it --rm --restart=Never test-pod --image=busybox --namespace default -- /bin/sh
```

```
# wget -T 2 -q -O- http://microservice-b.microservices.svc.cluster.local # This should time out or fail
```

Note: Network Policies require a Network Policy Controller (like Calico or Cilium) to be installed in your cluster. Minikube often includes one by default, but for EKS, you might need to install it separately (e.g., Calico).

Going Further: Next Steps for Your Kubernetes Journey

This guide provides a solid foundation for managing microservices on Kubernetes. To deepen your expertise and build more robust, production-grade systems, consider exploring these advanced topics:

- **CI/CD Integration:** Automate the deployment process of your microservices using tools like GitHub Actions, GitLab CI/CD, Jenkins, or Argo CD/Flux (GitOps). This ensures consistent and repeatable deployments.
- **Automated TLS with cert-manager:** Secure your ingress endpoints with automatically provisioned and renewed TLS certificates using cert-manager and Let's Encrypt.
- **Advanced Observability:**
 - **Distributed Tracing:** Implement distributed tracing with tools like Jaeger or Zipkin to visualize request flows across multiple microservices and pinpoint performance bottlenecks.
 - **Centralized Logging:** Set up a centralized logging solution (e.g., Elasticsearch, Fluentd, Kibana - EFK stack, or Loki, Promtail, Grafana - LGTM stack) to aggregate and analyze logs from all your pods.
- **Service Mesh (Istio/Linkerd):** Explore service mesh technologies to add advanced traffic management, security, and observability features without modifying your application code.
- **Chaos Engineering:** Introduce controlled failures into your system using tools like LitmusChaos or Gremlin to test resilience and identify weaknesses.
- **Cost Optimization:** Learn about Kubernetes cost optimization strategies, including right-sizing resources, using spot instances (on EKS), and leveraging Kubernetes cost management tools.
- **Custom Metrics for HPA:** Extend HPA to scale based on custom application-specific metrics exposed by Prometheus, rather than just CPU/Memory.

Phase 8: Cleanup (15 minutes)

It's important to clean up your resources, especially if you used EKS, to avoid incurring unnecessary costs.

Step 8.1: Uninstall Helm Charts

First, uninstall the applications and monitoring stack deployed via Helm.

```
# Navigate back to the main project directory
cd ~/kubernetes-microservices
```

```
# Uninstall microservices
helm uninstall microservice-a -n microservices
helm uninstall microservice-b -n microservices
```

```
# Uninstall Prometheus stack
helm uninstall prometheus-stack -n monitoring
```

```
# For Minikube, if you installed Minio with Helm
helm uninstall minio -n velero
```

Step 8.2: Delete Kubernetes Cluster

Now, delete your Kubernetes cluster.

- **For Minikube:**
minikube stop
minikube delete
- **For EKS:**
eksctl delete cluster -f ~/kubernetes-microservices/cluster-config.yaml

This command will delete all resources associated with your EKS cluster, including EC2 instances, security groups, and IAM roles. This may take 15-20 minutes.

Important: Also remember to delete the S3 bucket and IAM user/policy you created for Velero if you no longer need them to avoid ongoing AWS costs.

```
# Delete S3 bucket (be careful, this permanently deletes all contents)
aws s3 rb s3://<YOUR_VELERO_BUCKET_NAME> --force
```

```
# Delete IAM access keys (replace velero-user if you used a different name)
# First, list access keys to get the AccessKeyId
# aws iam list-access-keys --user-name velero-user
# Then delete the key
# aws iam delete-access-key --user-name velero-user --access-key-id
<ACCESS_KEY_ID>
```

```
# Delete IAM user
# aws iam delete-user --user-name velero-user

# Delete IAM policy
# aws iam delete-policy --policy-arn
arn:aws:iam::<YOUR_AWS_ACCOUNT_ID>:policy/VeleroBackupPolicy
```

Replace placeholders with your actual values.

Step 8.3: Remove Project Directory

Finally, clean up the project directory and any downloaded tools.

```
# Remove the main project directory
rm -rf ~/kubernetes-microservices

# Remove downloaded Velero CLI files
rm -rf velero-${VELERO_VERSION}-linux-amd64.tar.gz
rm -rf velero-${VELERO_VERSION}-linux-amd64

# Remove downloaded AWS CLI installer files (if you downloaded it)
rm -rf awscliv2.zip aws

# Remove VPA autoscaler clone (if you cloned it)
rm -rf /tmp/autoscaler
```

Congratulations! You have successfully deployed, monitored, auto-scaled, backed up, and cleaned up a Kubernetes microservices platform. This project demonstrates key skills for managing applications in a production-like Kubernetes environment.